Eliminating Redundant Recomputation in a Concurrent Recursive Frame Filter: Known Mechanisms and Ranked Fixes

TL;DR

- **Yes, this is a well-known problem.** The pattern you describe — concurrent consumers over a shared expensive computation where late arrivers cannot see in-flight (started-but-unpublished) work and therefore redundantly recompute it, with duplicates suppressed only at store time — is the classic **cache stampede** / **thundering herd** / **dog-pile** problem, and its canonical fix is **request coalescing** / **duplicate suppression** implemented as a **future/promise installed in the cache at compute start** (a.k.a. "singleflight", "leases", "memo-futures"). Your added twist — a strict recursive chain output[N]=f(source[N],output[N−1]) — makes it specifically analogous to **ffmpeg's frame-threading inter-frame dependency waits** (`ff_thread_report_progress` / `ff_thread_await_progress`).
- **The single highest-fit fix is a reservation/promise table keyed by frame number** (an "in-flight registry"): an activation atomically claims frame N at compute *start* by inserting a not-yet-ready placeholder (a `std::shared_future` or a condition-variable-backed slot) into your cache index; later activations that need N find the claim and **await** it (or compute their own single copy) instead of re-walking and recomputing. This moves the winner decision from store-time to claim-time (before the work is done) and drives your 18.75× overcompute toward ~1×. It must be paired with strict lock scope (never hold the cache mutex during compute) and bounded, deadlock-safe waits.
- **A second, complementary lever is admission/scheduling**: because the chain is genuinely serial, only ~1 frame of true forward progress exists at any moment, so most extra in-flight depth is intrinsically wasteful. Biasing service toward chain order ("only the successor of the newest published frame computes; others adopt or wait") or bounding useful in-flight depth caps the blast radius even without perfect coordination. Note VapourSynth's core *already* coalesces identical (node, frame) requests through its `allContexts` map — your waste exists precisely because your plugin computes predecessors internally, bypassing that mechanism.

Key Findings

1. The problem has established names in several literatures — and they all describe the same root cause.

- **Web/cache systems:** "cache stampede," "thundering herd," "dog-piling." The standard mitigation is **request coalescing** (also called request collapsing, request deduplication, or single-flight).
- **Go ecosystem:** the `golang.org/x/sync/singleflight` package. Its documented contract (pkg.go.dev): *"Package singleflight provides a duplicate function call suppression*

mechanism. Do executes and returns the results of the given function, making sure that only one execution is in-flight for a given key at a time. If a duplicate comes in, the duplicate caller waits for the original to complete and receives the same results." Internally a `call` object represents "an in-flight or completed singleflight.Do call," and `DoChan` appends duplicate callers to a shared channel list so they all receive one result — the concrete "install a placeholder at claim-start" pattern your reservation table would mirror.

- **JVM ecosystem:** future-based memoization — storing a `CompletableFuture` in the cache map at *compute start* (Caffeine's `AsyncLoadingCache` "establishes the hash table mappings immediately where an entry holds a CompletableFuture … subsequent calls for an entry obtains an existing future, which may be in-flight"), or `ConcurrentHashMap.computeIfAbsent` wrapping a `FutureTask`.
- **Facebook Memcache (Nishtala et al., 10th USENIX NSDI '13, "Scaling Memcache at Facebook"): leases** address both stale sets and thundering herds. Per the MIT 6.5840 course notes: *"mc gives just the first missing client a 'lease' … mc tells others 'try get() again in a few milliseconds'."* Leases are rate-limited — tokens are granted "only once every 10 seconds per key."
- **Video codecs:** ffmpeg frame-threading "progress" — the closest structural analogue to your recursive chain (deep-dive below).

2. Your defect is fundamentally "duplicate detection happening too late." Every literature converges on the same framing: the winner is decided at *publish/store* time (after the expensive work is finished) instead of at *claim/start* time (before it begins). Moving the coordination point earlier is the whole game — and it is exactly what your zero-waste fmParallelRequests observation already demonstrates.

3. VapourSynth's own core already solves this for normal graph edges — you are bypassing it. VapourSynth's thread pool keeps an `allContexts` map keyed on (node, frame); when a filter requests a frame, the core checks whether a context already exists — if so it adds the requester to that context's notification list, and only if new does it create a context and queue a task ("Context Deduplication"). This is textbook request coalescing, so the upstream getFrame for a given (node, n) runs once and all waiters are notified. Your plugin recomputes because it maintains its *own* internal output cache and computes predecessors *inside* getFrame rather than routing the output[N−1] dependency back through the core — so the core never sees two requests for the same frame to coalesce. Your reservation table is, in effect, re-implementing the core's coalescing for your private cache.

4. Parallel-prefix/scan reformulation is confirmed inapplicable to your f. Parallel scan requires an associative operator (or a linear/matrix recurrence). A nonlinear temporal IIR-like blend breaks associativity; the literature is explicit that non-linear recurrences "cannot use the parallel scan algorithm … due to the non-linearity breaking associativity, which typically forces sequential computation." (Linear recurrences *can* be scanned, and even linear IIR filters can be parallelized via matrix scan or pipelined look-ahead — but that requires linearity/time-invariance you do not

have.) Conclusion: do not try to parallelize the chain mathematically; pipeline it and eliminate duplicate work along it.

5. Blocking inside getFrame is historically dangerous but bounded-blocking is feasible. VapourSynth's documented model is request-then-return-NULL (arInitial → arAllFramesReady re-entry) precisely so a worker thread is freed rather than blocked. The ChangeLog records (r19): *"calls to the getFrame() function inside vapoursynth can never deadlock now, the thread handling is also slightly improved,"* and separately *"calling getFrame from a getFrameAsync callback no longer deadlocks, but you probably shouldn't have been doing it anyway."* That hardening covers the core's own calls, not a plugin that blocks on private state. Any internal blocking you add must be bounded, must never hold the cache mutex, and should block only on *your own* in-flight computes (guaranteed to make progress on an already-running plugin thread), never on a frame that can only be produced by a host thread your block is starving.

Details

The concrete mapping: what a "reservation"/"future"/"progress" is for your frame cache

Your cache index currently maps `frameNumber → storedFrame`. The fix widens the value to a small per-key state machine:

- **ABSENT** — not present, not claimed.
- **RESERVED / IN-FLIGHT** — some activation has claimed frame N and is computing it; the slot carries a waitable handle (a `std::condition_variable` + state flag, a Win32 event, or a `std::shared_future<FramePtr>`).
- **READY** — computed and stored (your current "stored" state).

The recovery walk changes from "find nearest present anchor, recompute the whole hole" to: *for each hole frame from the anchor upward — adopt if READY; if RESERVED, await the peer's handle (or compute a single own copy); only if ABSENT do I claim and compute it myself, then publish.* This is exactly the singleflight/memo-future pattern specialized to a linear dependency chain, and it recreates the fmParallelRequests behaviour (holes already adopted by execution time) *without* giving up compute parallelism — the claim becomes the serialization point, the compute does not.

Deep dive: ffmpeg frame-threading (the closest analogue) and why it does not deadlock

ffmpeg decodes frame N on one thread while frame N−1 is still being decoded on another; N needs motion-compensation references *into* N−1. Rather than recompute or fully serialize, ffmpeg uses **progress reporting**. Verbatim from the FFmpeg doxygen (`libavcodec/pthread_frame.c`):

- `ff_thread_report_progress(frame, progress, field)` — *"Notify later decoding threads when part of their reference picture is ready. Call this when some part of the picture is finished decoding. Later calls with lower values of progress have no effect."* FFmpeg

- `ff_thread_await_progress(frame, progress, field)` — *"Wait for earlier decoding threads to finish reference pictures. Call this before accessing some part of a picture, with a given value for progress, and it will return after the responsible decoding thread calls ff_thread_report_progress() with the same or higher value for progress."*

Progress is a monotonically increasing integer (typically decoded row count), guarded by a per-frame `progress_mutex` + `progress_cond`. Its deadlock-avoidance properties:

1. **Strict past-only dependency + bounded threads with fixed delay.** Threads are dispatched in decode order; ffmpeg's multithreading doc states "there is one frame of delay added for every thread beyond the first one." A thread only ever *awaits earlier* frames, never later ones, so the wait graph is a strict partial order (DAG) and cannot cycle. Fossies
2. **Progress is published incrementally, not at end.** Because progress is reported per-row *during* decode (via the `draw_horiz_band` hook), a waiter is released as soon as *its specific* reference region is ready — partial/progressive publication, not whole-frame publish-at-end.
3. **Setup handoff via `ff_thread_finish_setup`.** Context variables the next frame needs are updated *before* the heavy decode begins, and the next thread is released at that point, so the producer never holds a lock across the whole compute.

Portability to your plugin: the *mechanism* — a per-frame waitable progress/ready handle that late consumers await, over a strict past-only dependency — is directly portable and is essentially the reservation table above. What is **not** portable is ffmpeg's luxury of owning its own worker threads and dispatch order. You run on VapourSynth's threads and cannot guarantee the thread computing N−1 is scheduled before the thread waiting on it. This is the crux of your deadlock analysis: an ffmpeg-style await is safe only if the awaited work is guaranteed to be running or runnable on a thread you are *not* blocking. Two safe ways to obtain that guarantee: (a) block only on a compute an *already-running* plugin thread owns (bounded by your claim table, with a timeout + fall-back-to-compute if the claimant stalls); or (b) never block a worker at all — adopt-if-ready / reserve-if-absent, and otherwise compute your own copy, converting blocking into at-most-one redundant compute (the partial variant below). Note also that whole-frame temporal blending cannot consume a *partial* predecessor, so ffmpeg's per-row granularity collapses to binary (done/not-done) in your case.

The ranked candidate schemes (fit-first)

#1 — Reservation / in-flight registry with bounded await (memo-future / singleflight, specialized to the chain). BEST FIT.

- *How it works:* claim frame N at compute start by inserting an IN-FLIGHT placeholder under the cache mutex, release the mutex, compute, then transition to READY and notify waiters. A peer needing N takes the mutex, finds IN-FLIGHT, grabs the waitable handle, releases the mutex, and waits (with a timeout).

- *Where used in practice:* Go `singleflight` ("only one execution is in-flight for a given key at a time … a duplicate caller waits for the original to complete and receives the same results"); Caffeine `AsyncLoadingCache` (CompletableFuture installed immediately); Java `ConcurrentHashMap.computeIfAbsent` with a `FutureTask`; Facebook Memcache leases (NSDI '13).
- *Effectiveness:* eliminates essentially all of the 18.75× overcompute — the winner is decided before the work, so each hole frame is computed at most once and peers await. Also removes the memory churn from discarded duplicates.
- *Complexity/risk:* moderate. Cache lock must stay outside compute (you already know this). The finite host pool is the main hazard: bound every wait with a timeout and, on timeout, fall back to computing locally (steal-on-failure) so a dead/slow claimant cannot hang a worker forever. On claimant failure, transition the slot to ABSENT/ERROR and notify waiters so one re-claims. Priority inversion is minor in a single process and the timeout covers it.
- *Partial variant (recommended first step):* **adopt-if-ready-else-reserve-else-compute without blocking** — if you won't block a worker, an activation that finds IN-FLIGHT computes its own copy (accepting at most one duplicate per contended frame) rather than waiting. This alone collapses the *cascade* (the ~22-frame multi-checkpoint replans at depth 8) because each hole is claimed exactly once per walk even if occasionally computed twice.

#2 — Progress-based / progressive publication (ffmpeg pattern).

- *How it works:* publish frame N as "available for adoption" the instant it is READY (or at coarse sub-frame progress), and have the recovery walk await the nearest in-flight predecessor's readiness rather than recompute. For a whole-frame IIR blend, "progress" is binary, collapsing this into #1; ffmpeg's per-row granularity adds value only if a partial predecessor is consumable, which yours is not.
- *Effectiveness:* same near-elimination as #1 at whole-frame granularity. *Risk:* identical deadlock considerations; use bounded waits.

#3 — Route the recursive dependency back through VapourSynth's own coalescing (architectural fix).

- *How it works:* obtain output[N−1] via the host's request mechanism (which already deduplicates identical (node, frame) requests) instead of computing it internally.
- *Feasibility:* largely **infeasible as a pure graph edge** here. VapourSynth dependencies form a DAG fixed at creation, declared via `VSFilterDependency` arrays; a node cannot list itself, and the R76 changelog explicitly flags requesting frames from an undeclared node as a bug ("could cause memory leaks if an API4 filter requested frames from a node not properly declared as a dependency … just what MVTools v25&v26 did"). The supported way to do temporal feedback is internal per-instance state — which is exactly what forces your own cache. Value: a design lens confirming that scheme #1 is you re-implementing the core's coalescing for your private cache.

#4 — Admission control / dependency-aware scheduling (bias service toward chain order).

- *How it works:* since only the successor of the newest published frame makes true progress, add a policy: an activation for N whose predecessor is neither READY nor IN-FLIGHT either (a) claims and computes the *predecessor* first (walking the chain forward under reservations), or (b) parks briefly and re-checks; optionally bound useful in-flight depth to a small window around the frontier.
- *Where used:* OpenMP `task depend` / TBB flow-graph edges (a consumer task waits on the producer task instead of spawning duplicate work); ffmpeg's "one frame of delay per thread" dispatch discipline.
- *Effectiveness:* does not by itself reach 1× but sharply reduces waste by shrinking the recovery span and the number of concurrent walkers; pairs naturally with #1. Matches the requester's stated willingness to tolerate some out-of-order waste in exchange for much smaller numbers.
- *Risk:* low correctness risk; main risk is throughput loss from over-serialization.

#5 — Per-key locks / mutex-per-frame (coarse coalescing).

- *How it works:* a lock (or lock-striped set) keyed by frame number; the first activation to grab frame N's lock computes and stores, others block then find READY and adopt. This is the simplest coalescing and mirrors nginx `proxy_cache_lock` / the basic "distributed lock" stampede fix. Per the NGINX docs: *"When enabled, only one request at a time will be allowed to populate a new cache element … Other requests … will either wait for a response to appear in the cache or the cache lock for this element to be released, up to the time set by the proxy_cache_lock_timeout directive."*
- *Effectiveness:* eliminates duplicate compute like #1, but is strictly inferior: holding a per-frame lock across a long compute risks convoying and, on a finite host pool, is more deadlock-prone than a future/await with timeout. nginx's implementation is a cautionary example — its default `proxy_cache_lock_timeout` is 500 ms and, per independent analysis (nejc.blog, Nov 6 2024), it "adds up to 500ms to response time for locked requests," and cache locking is per-worker so requests can still leak. Use only if a full reservation table is deemed too invasive.

Why fmParallelRequests showed zero waste

Your observation is fully consistent with the analysis: with computes serialized, every recovery plan's holes are already READY ("adopted") by the time the plan executes — the store-time winner is decided before any peer starts the same work. A reservation table recreates that state while keeping compute parallelism, because the claim (not the compute) is the serialization point.

Recommendations

Stage 1 — Implement the reservation table (scheme #1), non-blocking variant first. Widen the cache index to {ABSENT, IN-FLIGHT, READY} with a per-slot waitable handle. In the recovery walk, for each hole frame: adopt if READY; if IN-FLIGHT, *for now compute your own copy* (partial variant) rather than block; if ABSENT, claim → compute → publish. This is a small, low-risk change that removes the multi-checkpoint cascade and should drop 8-in-flight overcompute from 18.75× toward a small constant. Benchmark duplicates at depths 2/4/8 against your current 197 / 575 / 3550.

Stage 2 — Add bounded await. Once the claim table is proven correct, upgrade IN-FLIGHT handling from "compute my own copy" to "await the claimant's handle with a timeout (a few hundred ms, or a small multiple of measured per-frame compute time), falling back to local compute on timeout." This should reach ≈1× overcompute. **Threshold to keep it:** if depth-8 duplicates don't fall below ~1× + a few percent, or if any wait hits the timeout under normal load, revert to the non-blocking variant — a timeout being hit means a host worker is being starved and you are near the deadlock hazard.

Stage 3 — Add admission bias (scheme #4) only if residual waste remains. If out-of-order arrivals still cause redundant *walks* (not computes), add the "compute the predecessor under reservation, walking forward" policy and/or bound in-flight depth to a small window around the published frontier. **Threshold:** adopt only if Stage 2 leaves >~5–10% redundant compute; otherwise the added serialization isn't worth the throughput cost.

Guardrails for every stage:

- Never hold the cache mutex during compute; the mutex protects only index transitions and handle publication.
- Every wait must be bounded and must have a compute fallback (steal-on-failure) so a dead/slow claimant cannot deadlock the finite host pool.
- On claimant error/abandonment, transition the slot to ABSENT/ERROR and notify waiters so exactly one re-claims.
- Treat checkpoints and scene-change floors as reservation boundaries: a fresh-start floor is just an ABSENT frame any activation may claim, so the chain restarts cleanly.

Caveats

- **Deadlock is the real risk, not correctness.** Correctness is already guaranteed by first-store-wins, and the reservation table preserves it (claim winner = store winner). The hazard introduced by *blocking* variants is starving VapourSynth's finite worker pool. ffmpeg avoids this only because it owns its threads and dispatches strictly in dependency order — a guarantee you lack. Bounded-await + compute-fallback is mandatory, not optional.

- **VapourSynth-specific blocking guidance is thin and partly inferred.** The documented safe model is request-then-return-NULL; there is no official blessing of long internal blocking inside getFrame. The ChangeLog hardening ("getFrame … can never deadlock now") covers the core's own calls, not plugin private-state blocking. No explicit developer statement was found sanctioning blocking on internal futures, so the non-blocking adopt-or-compute variant is the safer default and blocking should be gated behind the timeout guardrail.
- **Self-referential graph edges are not supported.** VapourSynth's dependency model is a creation-time DAG; a node cannot depend on its own output, which is precisely why recursive temporal feedback must live in per-instance internal state (and hence your own cache). Scheme #3 is therefore a design lens, not a shippable fix.
- **Whether output[N−1] is still resident is not guaranteed by the core.** VapourSynth's per-node cache is adaptive, can be disabled for certain request patterns (rpStrictSpatial / rpNoFrameReuse), and shrinks under memory pressure (R76 reworked cache-limit adherence). Your reservation table must not assume the host retains anything.
- **Parallel-prefix is out; pipelining is in.** Do not parallelize the recurrence mathematically (nonlinear f defeats scan). The realistic ceiling is a fully pipelined serial chain with zero duplicate work and lookahead limited to prefetching *source* frames, not parallelizing *output* frames.
- **Secondary-source numbers vary.** Effectiveness figures for singleflight-style coalescing quoted in web-cache blogs (e.g., "~90%+ origin reduction," "~95% duplicate reduction") are workload-dependent illustrations, not guarantees for your workload; your own depth-2/4/8 duplicate counts are the benchmark that matters.